SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	**ITA03**	**Course Title:**	**Mobile computing**
CO Assessed:	**CO4**	**Assessment:**	**AT2 – CASE STUDY**
Weightage:	**30 % of CIA**	**Total Marks:**	**100**
CO4 Statement:	***Identify the components and structure of mobile application for Android and Windows OS-based devices.***		
Student Name:	**KAVIYA M**	**Reg. No.:**	**192421050**
Section / Batch:	**SLOT D - 2024**	**Date:**	**18.08.2026**

Code of Conduct

I KAVIYA M – 192421050 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- **Duration: 60 minutes**
- **Number of questions : 01**

Faculty In-charge	**Course Coordinator**	**Head of Department**
Signature: _____	**Signature: _____**	**Signature: _____**
Date: _____	**Date: _____**	**Date: _____**

Title

Performance Improvement of Image Processing in Android Using Native Programming

1. Introduction

- A multimedia application on a smartphone may perform operations such as image filtering, resizing, enhancement, and compression.
- When these operations are implemented completely using Java/Kotlin high-level APIs, the application may work correctly but can have poor performance.
- Native programming using **C/C++ and Android NDK** can improve the performance of intensive image-processing tasks.

Objective: To study how Android Runtime and native programming work together and how native programming improves application performance.

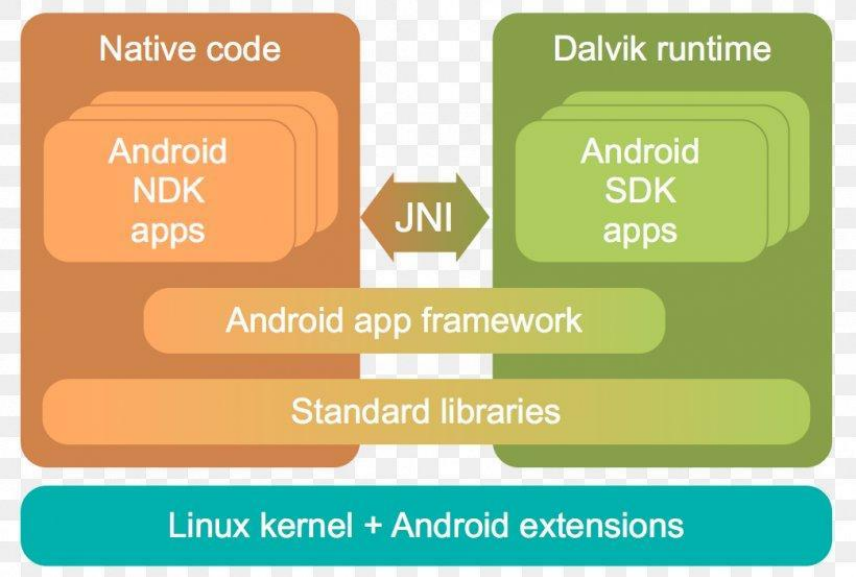


Figure 1: Overview of Android Image Processing and Native Programming

2. Problem Statement

The application performs image processing correctly, but it has poor performance.

Problems:

- Processing takes more time.
- CPU usage is high.
- Memory usage can increase.
- Application response becomes slower.

- Large images may cause delays.

Therefore, a better approach is required to improve performance.

3. Android Runtime and Native Programming

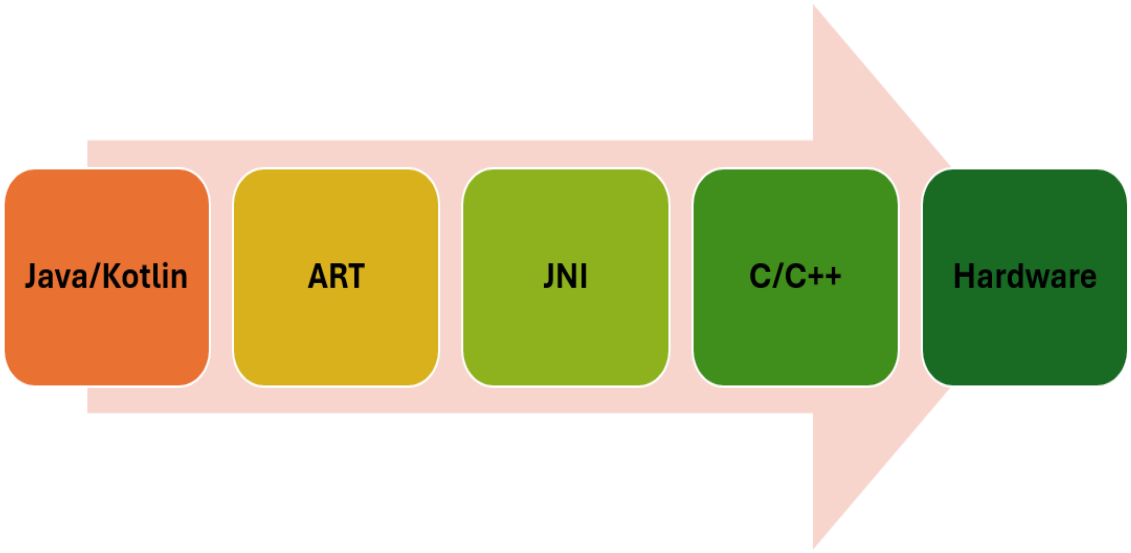
Android applications commonly use **Java/Kotlin** for application development.

The **Android Runtime (ART)** executes the application code and manages memory and garbage collection.

For intensive operations, **C/C++ native code** can be used through the **Android NDK**.

JNI (Java Native Interface) acts as a bridge between Java/Kotlin and C/C++.

Simple flow:



4. System Architecture

The proposed architecture contains five main layers:

1. **Android Application** – Java/Kotlin, UI and application logic.
2. **Android Runtime (ART)** – Executes code and manages memory.
3. **JNI** – Connects Java/Kotlin with native code.
4. **Native Layer (NDK)** – C/C++ performs intensive image processing.
5. **Mobile Hardware** – CPU, GPU, DSP and NEON provide processing power.

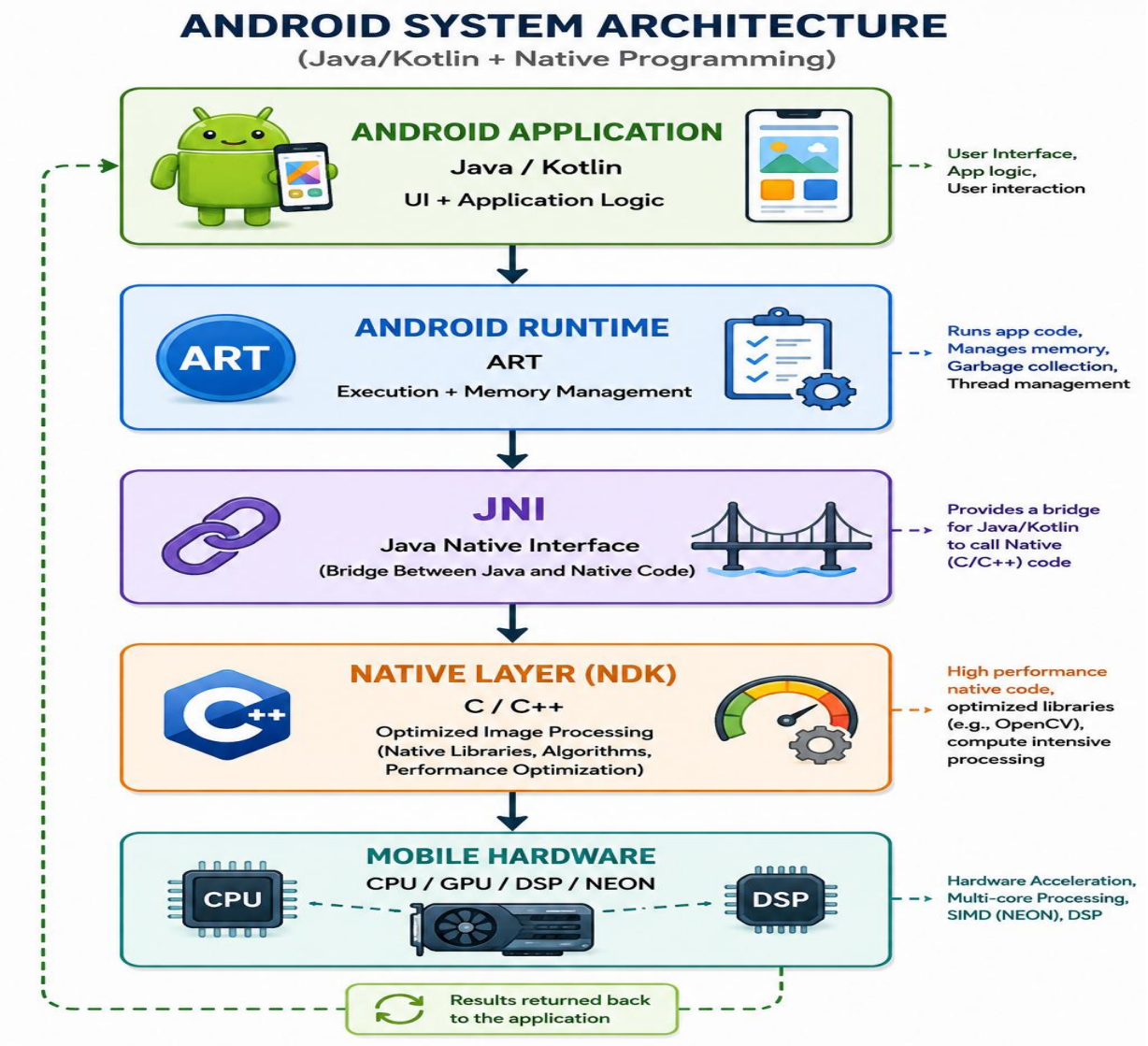


Figure 2: Android System Architecture with Native Programming

5. Role of Native Programming

Native programming improves performance in the following ways:

- **Faster processing:** C/C++ is suitable for computation-heavy operations.
- **Better memory control:** Native code provides more direct memory management.
- **Optimized libraries:** Libraries such as OpenCV can be used.
- **Hardware optimization:** Native code can use CPU and SIMD/NEON capabilities.
- **Multithreading:** Multiple CPU cores can be used for processing.

6. Java/Kotlin vs Native C/C++

Feature	Java/Kotlin	Native C/C++
Development	Easy	More complex
Performance	Good	High for intensive tasks
Memory control	Managed	More direct
Image processing	Suitable	Very suitable
Hardware access	More abstracted	More control
Development time	Less	More

7. Performance Comparison

An example performance comparison is shown below.

Metric	Java/Kotlin	Native C/C++
Processing Time	1200 ms	320 ms
CPU Usage	85%	42%
Memory Usage	150 MB	90 MB
Frame Rate	8 FPS	30 FPS

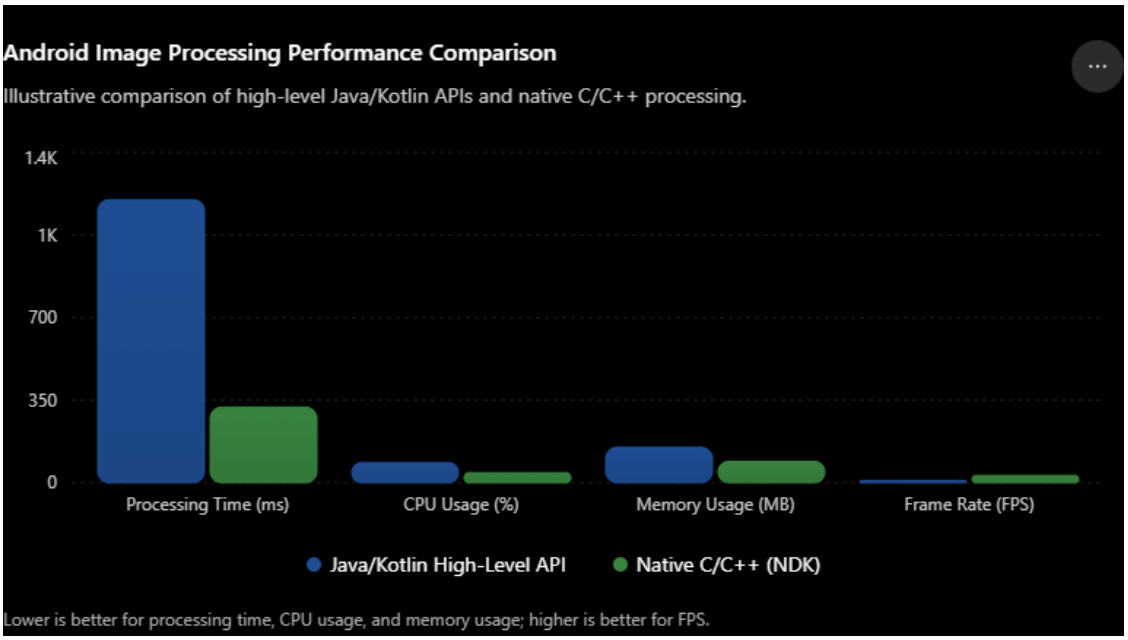


Figure 3: Performance Comparison Graph

Observation

Native C/C++ provides faster image processing with lower CPU and memory usage in this example.

8. JNI Flow

JNI provides communication between Java/Kotlin and native C/C++ code.

Processing Flow

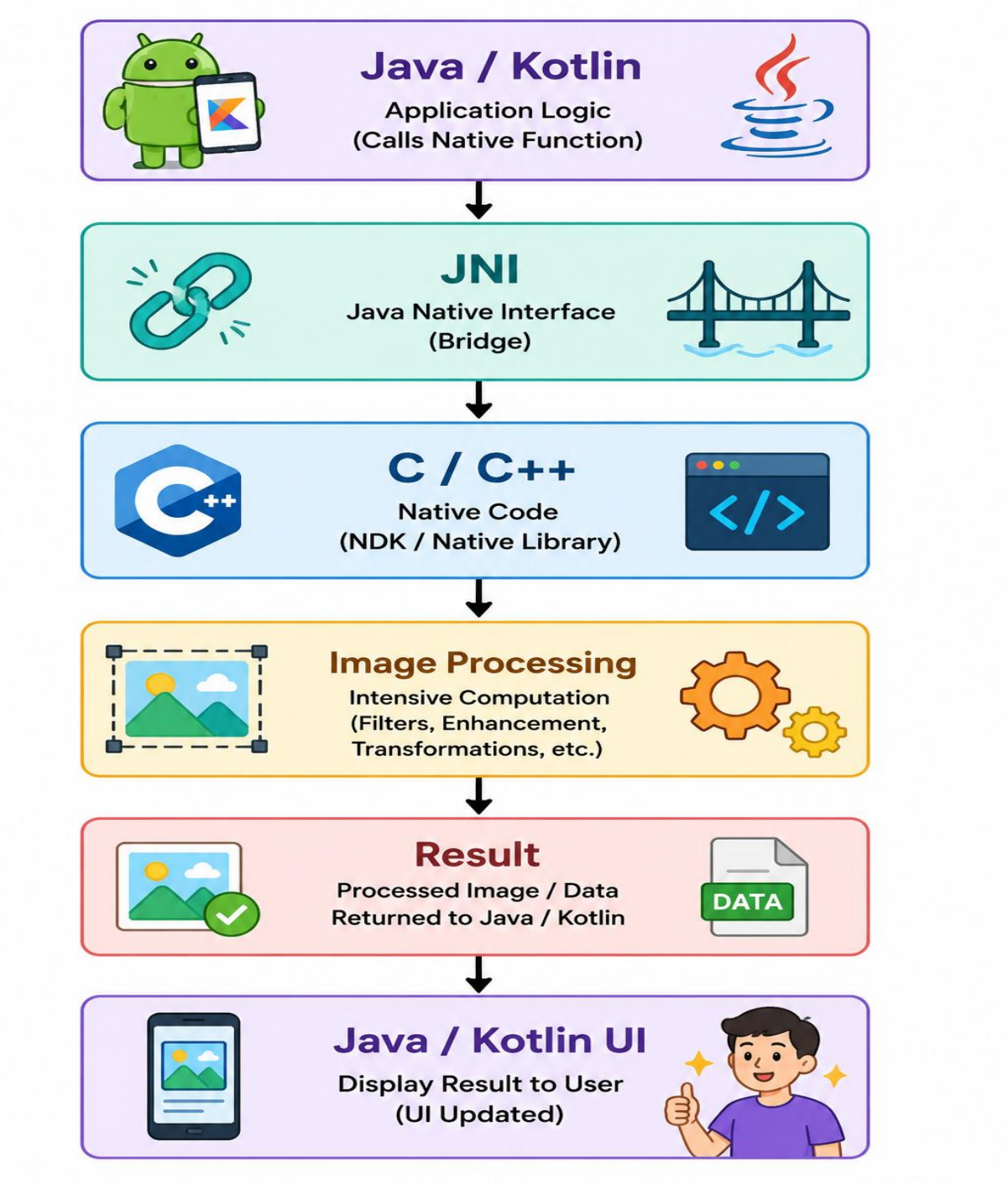


Figure 4: JNI-Based Image Processing Flow

9. Proposed Solution

A **hybrid approach** is recommended.

Java/Kotlin is used for:

- User interface
- Buttons and menus
- User interaction
- Application control

C/C++ is used for:

- Image filtering
- Image enhancement
- Image transformation
- Pixel-level calculations
- Other intensive operations

This provides a balance between **easy development and high performance**.

10. Advantages

- Faster image processing
- Lower processing time
- Reduced CPU usage
- Better memory utilization
- Higher frame rate
- Smoother application performance
- Better use of mobile hardware

11. Limitations

- C/C++ is more difficult to develop.
- Debugging is more complicated.
- Memory errors can cause crashes.
- JNI adds some complexity.
- Native code requires additional maintenance.

12. Expected Output

The proposed system is expected to:

- Process images faster.
- Reduce CPU usage.
- Reduce memory usage.
- Improve application response.
- Provide smoother multimedia performance.

Expected flow:

Input Image → Native Processing → Faster Output

13. Conclusion

Native programming can significantly improve the performance of Android applications that perform intensive image processing.

Java/Kotlin can be used for the user interface and normal application logic, while **C/C++ through Android NDK** can be used for computationally intensive operations.

JNI connects both layers and allows them to work together.

Therefore, a **hybrid Java/Kotlin + C/C++ approach** is a suitable solution for improving image-processing performance on smartphones.

14. Future Scope

Future improvements can include:

- GPU acceleration
- ARM NEON optimization
- Parallel processing
- AI/NPU acceleration
- Advanced image-processing libraries
- Real-time video and image processing